Planul proiectului, feature cu feature

Spring Boot 4.1.1 · Java 21 · MySQL 8.4 · Flyway · Spring AI 2.0.2 (MCP + Claude) · Kafka · Prometheus + Grafana · Tempo + ELK · Keycloak · Kong · React · doar Docker, pe EC2 · Rareș

Pornești de la codul tău, nu de la zero. Pe `7d89902` ai 337 de teste verzi și piesele grele: `AppointmentCommandServiceImpl` (`addAppointment`, `updateAppointment`, `confirmAppointment`, `cancelAppointment`, `completeAppointment`, scrise și testate), `AppointmentAvailabilityServiceImpl`, service-urile de frizeri și proceduri, entitățile (inclusiv `RecurringAppointment`), `GlobalExceptionHandler` cu `ApiError` și `DataSeeder`. Le lipsește drumul din HTTP: azi API-ul expune doar `POST /api/appointments` și `GET` availability. Planul acesta le leagă, pe rând.
Înlocuiește `PLAN_BACKEND.pdf` și `FEATURES.pdf`. Tabelele stau în `STRUCTURA_BD.pdf`, lângă acest plan. Conținutul din `FEATURES.pdf` rămâne, regrupat: F1 nou = vechile Frizeri, Proceduri, Program, Ore libere și statusurile programării, fără login; F3 nou = Conturile; Recurentele devin F4. Bug-urile din `CODE_REVIEW.pdf` se repară **în feature-ul căruia îi aparțin**: le găsești la „Repară”.
Scopul: să construiești singur un sistem real de programări la frizerie: întâi monolitul care chiar se folosește din HTTP, apoi baza cu istoric de schemă și conturile cu login, un job fără om în față (programările recurente), metrici live, un server MCP din care un agent vede salonul, un agent care **propune** reprogramări când un frizer lipsește (aprobate de client), același agent cu Claude prin Spring AI, Kafka și un serviciu de notificări desprins, tracing + ELK, Keycloak și Kong, plus interfața în React. Fiecare unealtă intră când rezolvă o problemă a salonului și e precedată de o lecție predată de Bogdan (rândul „Lecția de predat înainte” și tabelul de la final). F1–F2b sunt fără login; securitatea și testele cu login real încep la F3. Totul pe **Docker** (docker compose pe EC2), fără Kubernetes.
Fiecare feature merge de la A la Z, în aceeași ordine:
ce poate face utilizatorul → migrare `Vn__…sql` → model + repository → service → intrare (REST, tool MCP sau Kafka) → permisiuni → test (cu login real, de la F3) → log / metrică → verde în Actions → verificat
Un feature e gata doar când trece „Gata când” **acolo unde scrie la el**: F1 și F2 local, F2b pe EC2, de la F3 pe EC2 cu token din `/api/auth/login`; partea III local sau pe instanța de demo. Nu începi următorul înainte. Fiecare feature aduce **cel mult o unealtă nouă**. NEW = ceva ce nu ai făcut încă; înainte de feature, Bogdan îți predă lecția de dedesubt.

Unde ajungi (F14 + U5)

```
Internet → nginx (TLS, certbot) → ui React (booking-client, U1–U5)
                  → Kong DB-less (F14) → booking-api ── MySQL booking_api_db
booking-api ── /mcp (server MCP, F6–F7) ←── booking-agent (F8–F9, client MCP, fără bază)
                                                └──→ Claude API (Spring AI, F9)
booking-api ──→ Kafka (F10) ──→ booking-notifications (F11, booking_notifications_db)
                            └──→ booking-agent (pornește la NEEDS_RESCHEDULE)
topicuri: booking.appointment-events · booking.proposal-events
Keycloak (F13): oameni = login · agent = client credentials · ui = PKCE
nginx /auth → Keycloak · oauth2-proxy (F13) → Grafana · Kibana · Kafka UI
Prometheus + Grafana (F5) · Tempo + Elasticsearch / Kibana / Filebeat (F12) peste toate
```

Rețele Docker (F14): `edge` = nginx, Kong, oauth2-proxy, ui · `backend` = api, agent, notifications, MySQL, Kafka, Keycloak; în ambele doar Kong, oauth2-proxy și Keycloak (browserul ajunge la Keycloak prin nginx, pe `https://<domeniu>/auth`); aplicațiile fără `ports`. **Profiluri compose:** `core` (mysql, api, ui, nginx) · `obs` (prometheus, grafana) · `agent` · `messaging` (kafka, kafka-init, notifications) · `logs` (ELK + Tempo) · `auth` (keycloak + postgres, oauth2-proxy) · `edge` (kong).

Cine ce poate — enum-ul complet, de la F3

Permisiune	CUSTOMER	BARBER	ADMIN	AGENT	Folosită în
`USER_MANAGE`	–	–	✓	–	F3, F7
`BARBER_VIEW`	activi	activi	toți	toți	F3, F6
`BARBER_MANAGE`	–	–	✓	–	F3
`PROCEDURE_VIEW`	active	toate	toate	toate	F3, F6
`PROCEDURE_MANAGE`	–	ale lui	✓	–	F3
`SCHEDULE_VIEW`	–	al lui	toate	toate	F3, F6
`SCHEDULE_MANAGE`	–	al lui (program + time-off fără suprapunere)	✓	–	F3
`SCHEDULE_OVERRIDE`	–	–	✓ (time-off urgent peste programări)	–	F7
`OCCUPANCY_VIEW`	–	–	✓	–	F5, U3
`AVAILABILITY_VIEW`	✓	✓	✓	✓	F3, F6
`APPOINTMENT_CREATE`	pentru el (neblocat)	–	pentru orice client	–	F3, F4 (jobul = actor intern `SYSTEM`, nu rol)
`APPOINTMENT_VIEW`	ale lui	ale lui	toate	toate	F3, F6
`APPOINTMENT_RESCHEDULE`	ale lui	ale lui	toate	–	F3, F7
`APPOINTMENT_CANCEL`	ale lui	ale lui	toate	–	F3
`APPOINTMENT_PROCESS`	–	ale lui (confirm, complete)	toate	–	F3
`CUSTOMER_VIEW`	–	–	✓	–	F3
`CUSTOMER_MANAGE`	–	–	✓ (blocare)	–	F3
`RECURRING_VIEW`	ale lui	–	toate	–	F4
`RECURRING_MANAGE`	ale lui	–	toate	–	F4
`RECURRING_RUN`	–	–	✓	–	F4
`PROPOSAL_VIEW`	ale lui	ale lui	toate	toate	F7, F8, F9
`PROPOSAL_CREATE`	–	–	–	✓	F7, F8, F9
`PROPOSAL_DECIDE`	ale lui	–	toate	–	F7
`NOTIFICATION_VIEW`	ale lui	–	toate	–	F10, F11

„ale lui” = permisiunea e în token, iar service-ul verifică proprietarul. `AGENT` nu are `APPOINTMENT_RESCHEDULE` și nici `PROPOSAL_DECIDE`: agentul doar **propune**, decide clientul (F7). În F1–F2b nu există login; enum-ul intră întreg în F3. De la F13 rolurile vin din Keycloak; harta rol → permisiuni rămâne în API, aceeași.

I. Salonul merge (monolit)

O singură aplicație, `booking-api`. La final salonul se folosește din HTTP, are conturi, un job de noapte și ziua lui se vede în Grafana.

F1 Salonul se configurează și programările se gestionează din API

fără login încă · de la F3: ADMIN · BARBER · CUSTOMER

Administratorul adaugă, modifică și dezactivează frizeri; frizerul își pune procedurile cu prețul lui, programul săptămânal și perioadele libere; programarea se poate reprograma, confirma, anula și finaliza din HTTP, iar availability spune exact ce acceptă POST-ul.

Pornești de la: `AppointmentController` (doar POST), `AppointmentAvailabilityController`, `AppointmentCommandServiceImpl` (`#addAppointment` / `#updateAppointment` / `#confirmAppointment` / `#cancelAppointment` / `#completeAppointment`), `AppointmentAvailabilityServiceImpl#getAvailableSlots`, `AppointmentQueryServiceImpl`, `BarberCommandServiceImpl`, `BarberQueryServiceImpl`, `ProcedureCommandServiceImpl`, `ProcedureQueryServiceImpl`, `BarberSchedule`, `BarberTimeOff`, `BarberScheduleRepository`, `BarberTimeOffRepository`, `BarberScheduleStartTimeMustBeBeforeEndTime`, `GlobalExceptionHandler`, `ApiError`, `GlobalExceptionHandlerTest`, `DataSeeder` (`@Profile dev`), `booking-docs/openapi.yaml` · **Repară: M1, M2, M3, C3, N4, N5, N3 (partea din DataSeeder)**

Unealtă nouă: nimic nou, doar ce știi deja (controller, DTO, `@Valid`, `GlobalExceptionHandler`, `@Transactional`) · **Lecția de predat înainte:** niciuna.

1. **Bază.** Fără migrare încă: `ddl-auto: update` până la F2. Nicio entitate nu se schimbă în F1, deci schema de la finalul F1 devine exact `V1` în F2.
2. **Erori care spun adevărul (M1).** Două excepții noi: `AppointmentInPast` (400, „Appointment start time must be in the future”) și `BarberNotWorking` (409, „Barber is not working at the selected time”), aruncate din `addAppointment`, `validateBarberSchedule`, `validateBarberTimeOff` și `updateAppointment`. `AppointmentInvalidTimeInterval` rămâne doar pentru end ≤ start, `AppointmentTimeSlotUnavailable` doar pentru suprapunere.
3. **O singură validare pentru availability si POST (M2, M3).** O metodă comună (de ex. `BookingRules.validateBookable(barberId, procedureIds, date)`), chemată și de `getAvailableSlots`, și de `addAppointment`: frizer activ, proceduri existente, ale frizerului, active, fără duplicate, dată nu în trecut. Pentru azi, availability dă doar sloturile cu start după ora curentă. **Aceeași intrare → același cod, pe ambele rute.**
4. **Service-uri noi, mici.** `BarberScheduleService` (`setSchedule` = înlocuire totală a săptămânii, zile 1–7 unice, start < end; `getSchedule`) și `BarberTimeOffService` (create: end > start, 409 dacă se suprapune cu o programare ne-anulată; list între două date; delete). `BarberCommandServiceImpl.addBarber` creează `User` (role `BARBER`) + `Barber` într-o singură tranzacție, din `{email, firstName, lastName, phone}`; parola nu intră încă (vine în F3), `password_hash = "!"`.
5. **Procedurile frizerului (N5).** `Procedure` aparține unui singur frizer, deci rutele stau sub frizer și citesc cu `findByIdAndBarberId`; procedura altui frizer → 404, nu 200.
6. **Intrare.** `BarberController`: `GET /api/barbers`, `GET /api/barbers/{id}`, `POST /api/barbers`, `PUT /api/barbers/{id}`, `PATCH /api/barbers/{id}/deactivate`. `ProcedureController`: `GET/POST /api/barbers/{barberId}/procedures`, `PUT /{procedureId}`, `PATCH /{procedureId}/deactivate`. `BarberScheduleController`: `PUT/GET /api/barbers/{id}/schedule`. `BarberTimeOffController`: `POST/GET /api/barbers/{id}/time-off`, `DELETE /api/barbers/{id}/time-off/{timeOffId}`. `AppointmentController` completat (C3): `GET /api/appointments/{id}`, `GET /api/appointments?barberId=&date=`, `PUT /api/appointments/{id}` (reprogramare: `startTime`, `notes`), `PATCH /api/appointments/{id}/confirm`, `/cancel`, `/complete`. Fiecare body cu `@Valid`; toate erorile prin `GlobalExceptionHandler`.
7. **Contractul (N4).** Rămâne `ApiError {timestamp, status, error, message, path, details[]}` (există și e testat de 337 de teste); `openapi.yaml` se rescrie pe `ApiError` și pe rutele reale, fără cele care vin abia la F3 (login).
8. **Seeder (N3, partea lui).** Time-off-ul din `DataSeeder` se calculează față de azi (**vinerea** viitoare 12:00–13:00, ca orice luni să rămână fără pauză), nu 2026-10-26 scris de mână; `System.out.println` iese.
9. **Test.** `@WebMvcTest` pe fiecare controller nou, ca `GlobalExceptionHandlerTest`, câte un test pe fiecare cod din Reguli; teste de service pentru M1–M3. **Testele vechi care cer comportamentul de dinainte se actualizează, nu se șterg:** în `AppointmentCommandServiceImplTest`, `shouldRejectAppointmentInThePast` și `shouldRejectUpdatingAppointmentToPastDate` așteaptă acum `AppointmentInPast`, iar cele 6 pe program și time-off (`…DoesNotWorkThatDay`, `…StartsBeforeWorkingHours`, `…EndsAfterWorkingHours`, `…OverlapsBarberTimeOff` și cele 2 `…Updated…`) așteaptă `BarberNotWorking` (M1); în `BarberCommandServiceImplTest`, cele 3 teste `addBarber_*` se rescriu pe cererea nouă `{email, firstName, lastName, phone}` (fără `userId`, deci `addBarber_unknownUser_throwsException` devine „email existent → 409”). Un test din `AppointmentAvailabilityServiceImplTest` care acceptă o dată trecută sau o procedură inactivă se actualizează la fel (M2, M3). Restul celor 337 rămân neatinse.
10. **Postman.** `booking.postman_collection.json` în `booking-docs`: o cerere pe fiecare rută și pe fiecare cod de eroare din Reguli. Colecția e harta feature-ului: ce nu e în ea nu e verificat.
11. **Livrare.** Local, pe MySQL-ul tău de pe `localhost:3306`, cum rulezi aplicația azi; Docker apare abia în F2 (Testcontainers), EC2 în F2b.

Reguli

- Frizer: `firstName`, `lastName`, `phone` obligatorii (400); email valid și unic (409); frizer inexistent → 404; dezactivarea unui frizer cu programări viitoare ne-anulate → 409; frizerul inactiv nu mai are sloturi (409 „Barber not active”, la fel ca la POST).
- Procedură: nume unic per frizer (409), `durationMinutes` > 0, `price` ≥ 0 (400); procedura inactivă → availability și POST dau același 409 „Procedure not active”.
- Program: înlocuire totală; o zi 1–7 o singură dată (400); start < end (400). Time-off: end > start (400); peste o programare ne-anulată → 409.
- Programare: în trecut → 400 `AppointmentInPast`; duminică, în afara programului sau în time-off → 409 `BarberNotWorking`; suprapunere → 409 `AppointmentTimeSlotUnavailable`. Availability pe o dată trecută → 400.
- Statusuri: confirm doar din `BOOKED`, complete doar din `CONFIRMED`, cancel nu din `CANCELED` / `COMPLETED` → 409; reprogramarea revalidează slotul, ca POST-ul.

Gata când: local, pe datele din `DataSeeder`, într-o luni viitoare D: availability `barberId=1`, `procedureIds=1,2` (45 min) → 30 de sloturi, 09:00…16:15 · POST haircut la D 10:00 → 201, end 10:30, 50.00 · availability din nou → 26 de sloturi (dispar 09:30, 09:45, 10:00, 10:15) · POST D 16:45 haircut (30 min) → 409 „Barber is not working…", nu „already booked" · POST duminică → 409 `BarberNotWorking` · POST 2020-01-06T10:00 → 400 `AppointmentInPast` · availability `date=2020-01-06` → 400 · procedura 2 dezactivată prin PATCH → availability și POST dau același 409 · `PUT /api/barbers/1/procedures/2` cu `active=true` → availability revine la 26 (aceleași sloturi ca înainte de dezactivare) · PATCH confirm → `CONFIRMED`, complete → `COMPLETED`, cancel după complete → 409 · POST time-off D 10:00–11:00 → 409 · `GET /api/barbers` → 200 cu frizerul 1 · colecția Postman rulată integral, fără nicio cerere roșie · `mvn test` verde: toate cele 337 (cu cele 11 actualizate mai sus) + noile.

F2 Programările stau pe MySQL real, cu istoric de schemă NEW

fără login încă · local

Schema bazei e scrisă de mână și versionată, testele rulează pe MySQL adevărat, iar două rezervări în aceeași secundă pe același slot nu mai pot trece amândouă.

Pornești de la: `application.yaml` (`ddl-auto: update`, root/root, `show-sql true`), `application-test.yaml` (H2), `booking-api/pom.xml` (`java.version` 25), `AppointmentCommandServiceImpl#addAppointment` / `#updateAppointment`, `BarberRepository`, `GlobalExceptionHandler#handleDataIntegrity`, `DataSeeder`, entitățile de la finalul F1, `booking-db/` (gol) · **Repară: N2, N3** · NEW Flyway, Testcontainers, failsafe

Lecția de predat înainte: L1a „Baza versionată" — Flyway (`V1`, `ddl-auto: validate`, o migrare rulată nu se mai editează) · configurarea din variabile de mediu (`${DB_URL}`) · Testcontainers MySQL cu `@ServiceConnection` și maven-failsafe (`mvn verify`) · cursa: `@Lock(PESSIMISTIC_WRITE)` pe frizer + `READ COMMITTED` (de ce snapshot-ul din `REPEATABLE READ`, implicit pe MySQL, ascunde rezervarea celeilalte cereri) · testul cu două cereri în paralel (`CompletableFuture`, `@RepeatedTest`).

1. **Bază.** `booking-api/src/main/resources/db/migration/V1__schema.sql` (Flyway NEW) cu cele 9 tabele exact ca entitățile de la finalul F1 (tabelul „Migrările" din `STRUCTURA_BD.pdf`); `users.role` `VARCHAR(20)` și `appointments.status` `VARCHAR(30)`, nu `ENUM`: Hibernate generează pe MySQL `enum(...)`, deci V1 nu se copiază din DDL-ul generat și nici din dump-ul bazei locale; cu `ENUM`, `AGENT` (F7) și `NEEDS_RESCHEDULE` (F7) pică la INSERT cu „Data truncated", iar `validate` nu prinde asta. `ddl-auto: validate` NEW.
2. **Java (N3).** `java.version` 21 în `pom.xml`, ca în imaginea de build din F2b.
3. **Config (N3).** `application.yaml` citește `${DB_URL}`, `${DB_USER}`, `${DB_PASSWORD}` (variabile de mediu NEW); `show-sql false`; root/root iese din repo. `DataSeeder` trece pe `@Profile("demo")` și nu face nimic dacă există deja frizeri.
4. **Cursa (N2).** `addAppointment` și `updateAppointment` rulează cu `@Transactional(isolation = Isolation.READ_COMMITTED)` NEW și încep, **înainte de orice altă citire**, cu `BarberRepository.findByIdForUpdate` (`@Lock(PESSIMISTIC_WRITE)` NEW): două cereri pe același frizer se așteaptă una pe alta. Pe `REPEATABLE READ` (implicit pe MySQL 8) a doua cerere ar căuta suprapunerea într-un snapshot luat înainte de commit-ul primei și n-ar vedea rezervarea ei; pe `READ COMMITTED` fiecare SELECT vede ultimul commit, deci căutarea suprapunerii, făcută după lock, vede rezervarea celeilalte. `DataIntegrityViolationException` → 409 cu `ApiError`, nu 500.
5. **Test.** H2 iese. Cele 5 teste `@DataJpaTest` primesc `@AutoConfigureTestDatabase(replace = NONE)` și importă configurația Testcontainers (`MySQLContainer` cu `@ServiceConnection`) NEW, ca să ruleze pe `mysql:8.4` cu `V1`; `BookingApplicationTests` pornește la fel; `application-test.yaml` se șterge. Testele de integrare (`*IT`) rulează pe `mysql:8.4` din Testcontainers cu `@ServiceConnection`, prin `maven-failsafe-plugin` NEW; CI-ul (din F2b) rulează `mvn -B verify`. Un IT pornește două cereri pe același slot în paralel (`CompletableFuture` NEW), repetat de 20 de ori (`@RepeatedTest(20)` NEW): de fiecare dată unul 201, celălalt 409, un singur rând în `appointments` (pe H2 testul nu dovedește nimic, blocarea e diferită). Toate cele 337 + IT-ul cu două fire rulează în `mvn -B verify`.

Reguli

- O migrare rulată pe EC2 nu se mai editează: orice schimbare = migrare nouă, în același commit cu entitatea.
- Două cereri simultane pe același slot: exact una reușește (201), cealaltă 409; niciodată 500.

Gata când: local, pe o bază MySQL goală, cu `SPRING_PROFILES_ACTIVE=demo`: `SELECT version FROM flyway_schema_history` → un rând, 1 · `SHOW TABLES` → 9 tabele + `flyway_schema_history` · `SHOW CREATE TABLE users` / `appointments` → `role` și `status` sunt `varchar` · api repornit → pornește (`validate` trece), schema neschimbată · availability `barberId=1`, `procedureIds=1,2` într-o luni viitoare, pe datele demo → 30 de sloturi · IT-ul cu două cereri în paralel, la toate cele 20 de repetări → un 201 și un 409, un singur rând · `mvn -B verify` verde local, cu Testcontainers · grep după „root" în `application.yaml` → nimic.

F2b Salonul pe EC2 NEW

fără login încă · Rareș și Bogdan

Aplicația din F2 rulează pe EC2 în Docker, iar fiecare push pe `main` o testează și o livrează singur.

Pornești de la: aplicația de la finalul F2, `booking-ops/` (gol) · NEW imaginea Docker + compose + GitHub Actions, cu deploy prin pull de pe EC2

Lecția de predat înainte: L1b „Aplicația în container, pe EC2" — Dockerfile în 2 etape · compose cu healthcheck pe MySQL, profiluri și `.env` · GitHub Actions + imaginea în GHCR · EC2: security group și ssh · deploy prin pull: `cron` pe EC2 care rulează `deploy.sh`.

1. **Imagine.** `booking-api/Dockerfile` NEW în 2 etape (Maven + JDK 21 → JRE 21). `booking-ops/compose.yaml` NEW: `mysql:8.4` cu healthcheck NEW, api cu `depends_on: service_healthy`, profilul `core` NEW; la api, `environment` `TZ: Europe/Bucharest` și `JAVA_TOOL_OPTIONS=-Duser.timezone=Europe/Bucharest` (containerul JRE rulează altfel în UTC); `.env` NEW doar pe EC2 și local, `.env.example` în repo. În `.env` de pe EC2: `SPRING_PROFILES_ACTIVE=ec2,demo` (`ec2` = profilul de pe EC2: Swagger oprit de la F3, logurile ECS de la F4; `demo` = seeder-ul); în `.env.example` aceeași cheie. Baza de pe EC2 pornește goală.
2. **CI + livrare.** `.github/workflows/api.yml` NEW: `mvn -B verify` → imaginea în GHCR NEW (tag `latest` + SHA-ul commit-ului). Actions nu intră pe EC2: **EC2 își trage singur imaginea**. Pe EC2, `booking-ops/deploy.sh` (`git pull` + `docker compose pull` + `docker compose up -d`, cu ieșirea în `~/deploy.log`) rulat de `cron` NEW la 2 minute; o singură dată, de mână: `docker login ghcr.io` cu un token doar de citire. `up -d` recreează doar containerele cu imagine nouă. Profilurile le citește din `.env` de pe EC2: `COMPOSE_PROFILES=core` (fără el, `up -d` nu pornește nimic).

Reguli

- Până la F3 API-ul nu are login: security group-ul EC2 NEW deschide portul doar pentru IP-urile lui Rareș și Bogdan.
- Portul 22 doar pentru IP-ul lui Rareș; niciun pas din Actions nu folosește ssh, deci runner-ele GitHub n-au nevoie de acces la EC2.
- Secretele (parola MySQL) doar în `.env`; în repo doar `.env.example` cu valori goale.

Gata când: pe EC2, fără token: `docker compose ps` → mysql healthy, api up · `SELECT version FROM flyway_schema_history` → un rând, 1 · `SHOW TABLES` → 9 tabele + `flyway_schema_history` · api repornit → pornește (`validate` trece), schema neschimbată · availability `barberId=1`, `procedureIds=1,2` într-o luni viitoare, pe datele demo → 30 de sloturi · două POST simultane la luni L2 10:00 (L2 = o luni viitoare; F3 și F4 folosesc alte zile) (curl în paralel) → un 201 și un 409, `SELECT COUNT(*)` pe acea oră = 1 · Actions verde pe `mvn -B verify` cu Testcontainers · un push pe `main` → după ce Actions e verde, în cel mult 2 minute `docker compose ps` pe EC2 arată api recreat pe imaginea nouă, iar `~/deploy.log` are linia rulării.

F3 Contul meu: client, frizer, administrator NEW

vizitator · CUSTOMER · BARBER · ADMIN

Vizitatorul își face cont de client și se loghează; clientul își rezervă și își vede programările, frizerul își confirmă ziua, administratorul face conturile de frizer și blochează clienții. Fiecare vede și schimbă doar ce e al lui.

Pornești de la: `User`, `Role` (`CUSTOMER`, `BARBER`, `ADMIN`), `UserCommandServiceImpl#addUser` / `#changePassword`, `ChangePasswordRequest`, `UserCreateRequest`, `CustomerCommandServiceImpl`, `CustomerQueryServiceImpl`, `Customer.blocked`, controllerele din F1, testele `*IT` din F2 · **Repară: N1** · NEW Spring Security + JWT (cu TLS prin nginx + certbot, condiția deschiderii publice)

Lecția de predat înainte: L2 „Spring Security cu JWT” — `SecurityFilterChain` stateless (ce e public, ce cere token) și `PasswordEncoder` BCrypt · emiterea și validarea JWT într-un filtru (token 1 oră, authorities din token) · permisiunile în service: `Actor`, `access.require`, proprietarul, 401 vs 403 · teste cu login real: helperul register → login → token și `@TestConfiguration` pentru slice · nginx + certbot: TLS pe domeniu înainte de deschiderea publică.

1. **Bază.** `V2__accounts.sql`: parolele care nu sunt BCrypt devin `"!"` (de neutilizat). `Role` primește `AGENT` (contul se face în F7).
2. **Permisiuni.** Enum-ul `Permission` complet din tabelul „Cine ce poate” (inclusiv cele folosite abia la F4, F5, F7, F10) și o singură hartă `Role` → permisiuni, citită de `getAuthorities()`. JWT-ul NEW conține `sub` = email, `role` și `permissions` (lista din hartă); filtrul citește authorities din token. Fiecare permisiune cerută în service există în enum și o primește cine trebuie.
3. **Service.** `register` creează `User` (`CUSTOMER`) + `Customer` într-o tranzacție, orice rol ar trimite clientul. Un `CommandLineRunner` (ca `DataSeeder`) creează `ADMIN` din `ADMIN_EMAIL` / `ADMIN_PASSWORD`, dacă nu există. `addUser`, `changePassword` și `addBarber` salvează parola prin `PasswordEncoder` NEW (N1); `BarberCreateRequest` primește `password` (min. 8).
   Proprietarul, într-un singur loc: o metodă care decide accesul la o programare (`ADMIN` trece; `CUSTOMER` doar ale lui; `BARBER` doar ale lui). F6 și F7 o refolosesc. Service-urile primesc un `Actor` NEW (`record Actor(Kind kind, Long userId, Long customerId, Set<Permission> permissions)`, `Kind` = `USER` | `SYSTEM`), construit din token în controller. `SYSTEM` nu e rol, nu intră în `Role` și nu poate veni din HTTP.
4. **Intrare.** `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`, `POST /api/users` (`USER_MANAGE`: conturi BARBER / ADMIN), `PUT /api/users/{id}/password` (`USER_MANAGE`), `GET /api/customers` (`CUSTOMER_VIEW`), `PATCH /api/customers/{id}/blocked` (`CUSTOMER_MANAGE`), `GET /api/appointments/me`. Permisiunea se verifică în service, nu pe rută: fiecare metodă publică de service chemată dintr-un controller sau dintr-un tool începe cu `access.require(actor, Permission.X)` NEW, iar `Actor` poartă lista de permisiuni din token. **Actorul intern, o singură regulă (aceeași în F4):** `Actor.SYSTEM` are exact `{APPOINTMENT_CREATE}` și trece prin `access.require` ca oricine; sare doar peste verificarea de proprietar, nu peste `BookingRules` și nici peste `blocked`. Singura metodă chemată cu `SYSTEM` e `addAppointment`. La `POST /api/appointments`, `CUSTOMER` rezervă pentru el (`customerId` din token); doar `ADMIN` trimite `customerId` (și `SYSTEM`, cu clientul seriei, în F4).
5. **Test. Se schimbă și testele unitare de service**, nu doar cele pe endpoint-uri: fiecare metodă primește acum un `Actor`, deci apelurile din toate cele 11 clase `*ServiceImplTest` (Appointment, Barber, Procedure, Customer, User) și din testele de service din F1 primesc un actor din `ActorFixtures` (`admin()`, `customer(id)`, `barber(id)`), construit în test, fără Spring și fără token; per service, câte un test nou „fără permisiune → 403” și „programarea altuia → 403”; `BarberCommandServiceImplTest` și `UserCommandServiceImplTest` verifică și că parola trece prin `PasswordEncoder` (mock). Toate testele pe endpoint-uri devenite protejate trec pe token din helperul register → login, **fără `@WithMockUser`** (altfel CI-ul pică). Testele de controller din F1 (`@WebMvcTest`) devin `*IT`: `@SpringBootTest` + MockMvc pe Testcontainers din F2, cu token din helper. `GlobalExceptionHandlerTest` rămâne slice pe `ProbeController`, cu `/probe/**` permis doar în configurația de test a slice-ului (`@TestConfiguration` NEW cu `SecurityFilterChain permitAll`): testează maparea excepțiilor, nu accesul. Matricea cu conturi reale: admin, client A, client B, frizer 1, frizer 2, pe fiecare rută.
6. **Livrare.** `JWT_SECRET_KEY`, `JWT_EXPIRATION=3600`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` în `.env` pe EC2. Swagger dezactivat în profilul `ec2`. Security group-ul se deschide public doar pe 80/443, prin nginx NEW (profilul `core`) cu certificat certbot NEW pe domeniu; API-ul rămâne în spatele nginx, parolele și tokenurile nu circulă în clar.

Reguli

- Email valid și unic (409); parolă min. 8 (400); login greșit → 401, același mesaj pentru email inexistent și parolă greșită; parola nu apare în niciun răspuns.
- Fără token → 401; cu token, fără permisiune sau pe programarea altuia → 403 cu `ApiError`.
- Client blocat → 403 „Customer is blocked” la rezervare; programările lui existente rămân.
- Tokenul expiră după 1 oră (exp – iat = 3600).

Gata când: pe EC2, prin `https://<domeniu>`, pornind de la datele din F2b: adminul (din `.env`) setează parola frizerului 1 prin `PUT /api/users/{userIdFrizer1}/password` și creează frizerul 2 prin `POST /api/barbers` (cu `password`); clientul A și clientul B își fac cont prin register · register cu `"role":"ADMIN"` → contul creat e `CUSTOMER` · login greșit → 401 · `/api/auth/me` fără token → 401 · login cu adminul din `.env` → token cu exp – iat = 3600 · `POST /api/barbers` ca CUSTOMER → 403 · clientul A rezervă haircut luni D3 (alta decât L2) 10:00 → 201, 50.00, end 10:30 · clientul B face cancel pe ea → 403 · frizerul 1 face confirm → `CONFIRMED`; frizerul 2 → 403 · adminul îl blochează pe B → B rezervă → 403 · adminul îl deblochează (`PATCH /api/customers/{id}/blocked`, `blocked=false`) → B rezervă → 201 · `SELECT password_hash` → toate încep cu `$2` sau sunt `"!"` · Actions verde: testele de endpoint pe token, testele unitare de service pe `Actor`, niciun `@WithMockUser`.

F4 Programarea mea recurentă: la 2 săptămâni, aceeași oră NEW

CUSTOMER · ADMIN · jobul de noapte

Clientul își face o serie (de exemplu la 2 săptămâni, marți la 10:00, tuns); în fiecare noapte un job generează programările seriei pe următoarele 28 de zile, sare peste cele care nu încap și spune de ce.

Pornești de la: `RecurringAppointment`, `RecurringAppointmentRepository`, `Appointment.recurringAppointment`, `AppointmentCommandServiceImpl#addAppointment` (aceeași validare, cu actorul `SYSTEM`), metoda de proprietar din F3 · NEW logare structurată (MDC cu id-ul rulării, JSON pe EC2)

Lecția de predat înainte: L3 „Joburi programate și loguri care se pot căuta” — `@Scheduled` + `@EnableScheduling` · `Clock` injectat (ca testul să-și aleagă „azi”) · Logger SLF4J și nivelurile INFO / WARN / DEBUG · MDC cu `runId` și `seriesId` · `logging.structured.format.console=ecs` în profilul `ec2`.

1. **Bază.** `V3__recurring.sql`: `recurring_appointment_procedures`; pe `appointments` coloana `occurrence_date DATE NULL` (o completează doar jobul, apoi nu se mai schimbă: `updatable = false` în entitate) și UQ `(recurring_appointment_id, occurrence_date)`.
2. **Model.** `RecurringAppointment` primește lista de proceduri; ziua săptămânii vine din `startDate`.
3. **Service.** `RecurringAppointmentService`: create (validările din Reguli), list, deactivate și `runNow(actor)` = `access.require(actor, RECURRING_RUN)`, apoi `RecurringGenerator.run()`. `RecurringGenerator.run()` e o componentă internă, fără `Actor` și fără `access.require`, chemată doar de `@Scheduled` NEW și de `runNow`: citește seriile active direct din `RecurringAppointmentRepository` și, pentru fiecare apariție din următoarele 28 de zile care nu există încă, cheamă `addAppointment(Actor.SYSTEM, …)` cu clientul seriei, după regula actorului intern din F3 (doar `APPOINTMENT_CREATE`; fără verificarea de proprietar, dar cu `BookingRules` și `blocked`: client blocat → apariția sărită, motiv `CUSTOMER_BLOCKED`). Apariția există = există un rând cu (seria, `occurrence_date`), indiferent de status și de oră. Ce nu încape (frizer care nu lucrează, slot ocupat, procedură inactivă) se sare, cu motivul; un rezumat `{created, skipped}`.
4. **Clock.** Un bean `Clock.system(ZoneId.of("Europe/Bucharest"))` NEW, folosit de toate service-urile (inclusiv `canceled_at` din F5): `LocalDateTime.now()` devine `LocalDateTime.now(clock)`, ca testele să fixeze „azi” și pe EC2 ora să fie a României.
5. **Intrare.** `POST/GET /api/recurring-appointments`, `PATCH /{id}/deactivate` (`RECURRING_MANAGE` / `RECURRING_VIEW`); `POST /api/admin/recurring/run` (`RECURRING_RUN`, prin `runNow`) pentru demo; `@Scheduled(cron = "${booking.recurring.cron:0 0 2 * * *}")` NEW + `@EnableScheduling`.
6. **Logare.** Fiecare rulare primește `runId`, fiecare serie `seriesId`, ambele în MDC NEW. Prin Logger-ul SLF4J NEW: un INFO per rulare (serii, create, sărite, durata), un WARN per apariție sărită cu motivul, DEBUG per apariție creată. Pe EC2 logurile ies JSON (`logging.structured.format.console=ecs` NEW).
7. **Test.** IT cu `Clock` fix și login real: seria, rularea, a doua rulare, conflictul.

Reguli

- `intervalWeeks` între 1 și 8; `endDate` după `startDate` și cel mult 6 luni (400); `startDate` în viitor (400); clientul blocat nu face serii (403).
- Seria aparține unui singur frizer, iar procedurile sunt ale lui (aceleași reguli ca la POST).
- A doua rulare nu creează nimic (UQ pe serie + `occurrence_date`); jobul nu se oprește la prima apariție sărită.
- O apariție reprogramată sau anulată nu se recreează.

Gata când: pe EC2, cu token de client A: serie săptămânală luni 10:00, haircut, de la luni D4 la D4 + 8 săptămâni (D4 ≠ D3, după L2 și D3, fără programări la 10:00 în D4 … D4 + 21) · clientul B rezervă înainte, de mână, D4 + 14 la 10:00 · seria lui A, rezervarea lui B și rularea manuală se fac toate în aceeași zi D4 – 1, după rularea automată de la 02:00 (sau cu jobul automat oprit: `booking.recurring.cron` pus pe `"-"` în `.env` pe durata verificării) · admin: `POST /api/admin/recurring/run` în ziua D4 – 1 → `{created: 3, skipped: 1}` (D4, D4 + 7, D4 + 21; D4 + 14 sărită, slot ocupat) · a doua rulare → `{created: 0, skipped: 1}` · `GET /api/appointments/me` la A → 3 programări cu `recurringAppointmentId` · în logurile JSON de pe EC2 filtrezi după `runId` și vezi o linie INFO și un WARN cu motivul.

F5 Ziua salonului, live NEW

ADMIN (în Grafana)

Administratorul vede în Grafana, în timp real, cât de ocupat e fiecare frizer azi, câte programări sunt pe fiecare status, câte anulări târzii au fost și dacă jobul de noapte a mers.

Pornești de la: `AppointmentRepository`, `BarberScheduleRepository`, `AppointmentCommandServiceImpl#cancelAppointment` / `#confirmAppointment` / `#completeAppointment`, `RecurringGenerator` din F4, `booking-ops/compose.yaml` din F2b · NEW Micrometer + Prometheus + Grafana în compose

Lecția de predat înainte: L4 „Metrici cu Micrometer și Grafana” — Actuator și `/actuator/prometheus` pe un port de management nepublicat (`management.server.port`, `SecurityFilterChain` separat cu `EndpointRequest.to("prometheus")`) · Counter, Gauge, Timer (cu tag-uri) · Prometheus în compose care citește aplicația · Grafana cu sursa de date și dashboard-ul JSON încărcate automat · tunelul ssh (`ssh -L`) spre Grafana de pe EC2, până la F13.

1. **Bază.** `V4__appointment_lifecycle.sql`: `confirmed_at`, `completed_at`, `canceled_at`, `canceled_by_role` pe `appointments`; service-urile de status le completează.
2. **Calcul.** `OccupancyCalculator`: pentru un frizer și o zi, minutele programate ne-anulate / minutele din program; teste unitare pe zi fără program, zi plină, anulare.
3. **Intrare.** `GET /api/admin/occupancy?date=` (`OCCUPANCY_VIEW`) → `[{barberId, bookedMinutes, workMinutes, ratio}]`, calculat de `OccupancyCalculator`, aceeași valoare ca gauge-ul.
4. **Metrici.** Micrometer NEW: Gauge `booking_occupancy_ratio{barber}` pentru azi · Counter `booking_appointments_total{status}` la fiecare tranziție · Counter `booking_late_cancellations_total` (anulare cu < 2 h înainte de start) · Timer `booking_availability_seconds` · Counter `booking_recurring_skipped_total{reason}` din F4.
5. **Ops.** Prometheus NEW și Grafana NEW în compose, profilul `obs` (pe EC2 `COMPOSE_PROFILES=core,obs`); datasource + dashboard în `booking-ops/grafana/`, încărcate automat. Grafana e publicată doar pe EC2-ul însuși (`ports: "127.0.0.1:3000:3000"`), cu parola de admin din `.env` (`GF_SECURITY_ADMIN_PASSWORD`); o deschizi din laptop prin tunel ssh NEW: `ssh -L 3000:localhost:3000 ubuntu@<ip-EC2>`, apoi `http://localhost:3000`. Așa rămâne până la F13, când trece în spatele nginx + oauth2-proxy și portul iese. Actuator NEW: `/actuator/prometheus` se vede doar din rețeaua compose, nu public: `management.server.port: 8081`, nepublicat în compose (fără `ports`), deci doar Prometheus îl vede prin rețeaua compose; pe portul de management `/actuator/prometheus` e `permitAll` (`EndpointRequest.to("prometheus")` într-un `SecurityFilterChain` separat).
6. **Test.** Unitare pe `OccupancyCalculator`; un IT care verifică un contor după o anulare făcută cu login real; ocuparea cu login real: ADMIN → 200, CUSTOMER → 403.

Reguli

- Ocuparea = minute programate (`BOOKED`, `CONFIRMED`, `COMPLETED`) / minute de program în ziua aceea; zi fără program = 0, nu împărțire la 0.
- Anulare târzie = `canceled_at` mai târziu decât `start_time` – 2 h.

Gata când: pe EC2, frizerul 1, azi (zi lucrătoare, program 09:00–17:00 = 480 min, fără alte programări la frizerul 1: nu o luni din seria din F4), cu token (rezervările le faci înainte de 09:00, iar anularea celei de la 11:00 între 09:00 și 11:00, ora României): haircut 09:00 (30) + Haircut and Beard 11:00 (45) + Haircut and Beard 14:00 (45) = 120 min → Grafana (prin tunelul ssh, `http://localhost:3000`) arată 25% · `GET /api/admin/occupancy?date=` azi ca admin → `ratio` 0.25 la frizerul 1; ca client → 403 · clientul anulează 11:00 → 75 / 480 = 15,6% · anularea s-a făcut cu mai puțin de 2 h înainte → `booking_late_cancellations_total` crește cu 1 · curl pe `https://<domeniu>/actuator/prometheus` → 404 / 401; pe IP-ul EC2:8081 → connection refused / timeout.

II. MCP — piesa centrală

Salonul devine ceva ce poate folosi un agent: întâi doar citind, apoi propunând reprogramări pe care le aprobă clientul. Primul microserviciu, `booking-agent`, apare aici.

F6 Salonul văzut de un agent NEW

orice client MCP cu token (BARBER, ADMIN, apoi AGENT)

Un client MCP (MCP Inspector, mai târziu `booking-agent`) se conectează cu un token și întreabă de frizeri, proceduri, program, time-off, sloturi libere, programări și tiparul de vizite al unui client; vede exact cât vede utilizatorul tokenului.

Pornești de la: `BarberQueryServiceImpl`, `ProcedureQueryServiceImpl`, `BarberScheduleService` și `BarberTimeOffService` din F1, `AppointmentAvailabilityServiceImpl`, `AppointmentQueryServiceImpl`, metoda de proprietar din F3, filtrul JWT din F3 · NEW server MCP (Spring AI 2.0.2, `spring-ai-starter-mcp-server-webmvc`)

Lecția de predat înainte: L5 „Server MCP cu Spring AI" — ce e MCP (tool-uri, Streamable HTTP) · `@Tool` și `@ToolParam`, descrieri scrise pentru un model · MCP Inspector · endpoint-ul `/mcp` în spatele filtrului JWT.

1. **Dependență.** `spring-ai-bom` 2.0.2 + `spring-ai-starter-mcp-server-webmvc` NEW; build real pe Boot 4.1.1 **înainte de primul tool** (confirmă perechea 2.x ↔ 4.x).
2. **Tool-uri** (`@Tool` + `@ToolParam` NEW), în pachetul `mcp/`: `listBarbers`, `getBarberProcedures(barberId)`, `getBarberSchedule(barberId)`, `getTimeOffs(barberId, from, to)`, `getAvailability(barberId, date, procedureIds)`, `getAppointments(barberId, date)`, `getCustomerVisitPattern(customerId)` = zi + oră ale ultimelor 10 programări `COMPLETED`, fără nume și telefon.
3. **Regula tool-urilor.** Un tool = un apel la un service existent; fără repository, fără logică în tool.
4. **Permisiuni.** `/mcp` în spatele aceluiași filtru JWT; permisiunea și proprietarul se verifică în service, deci REST și MCP văd la fel.
5. **Fără migrare:** nicio entitate nu se schimbă.
6. **Log / metrică.** Fiecare apel: tool + utilizator într-o linie de log și Counter `booking_mcp_calls_total{tool}`.

Reguli

- Limite pe parametri: perioada din `getTimeOffs` ≤ 31 de zile; `getAppointments` pe o singură zi.
- Eroarea de acces din service ajunge la client ca eroare de tool, nu ca date goale.

Gata când: pe EC2, o luni viitoare D6 fără programări la frizerul 1 (în afara seriei din F4): clientul A rezervă haircut la 10:00 · MCP Inspector cu token de frizer 1 listează 7 tool-uri · `getAvailability(1, D6, [1,2])` → 26 de sloturi, la fel ca REST · `getAppointments(2, D6)` cu token de frizer 1 → eroare de acces · `getTimeOffs(1, …)` cu token de client → eroare de acces · fără token → 401 · contorul `booking_mcp_calls_total` crește în Grafana.

F7 Frizerul lipsește: propun → clientul aprobă → se reprogramează NEW

ADMIN · AGENT · CUSTOMER

Frizerul anunță dimineața că lipsește; administratorul pune un time-off urgent și programările lui din ziua aceea rămân de reprogramat. Agentul propune pentru fiecare o oră nouă; clientul acceptă sau refuză, iar acceptarea mută programarea pe aceeași cale ca o reprogramare făcută de mână.

Pornești de la: `BarberTimeOffService` din F1, `AppointmentCommandServiceImpl#updateAppointment`, `AppointmentStatus`, `AppointmentAvailabilityServiceImpl`, tool-urile din F6, `POST /api/users` din F3 (contul `AGENT`) · NEW tool-uri MCP de scriere (propunere → aprobare de un om → aceeași cale de execuție)

Lecția de predat înainte: L6 „Agentul propune, omul decide" — tool de scriere care doar salvează · statusurile propunerii (`PROPOSED` → `ACCEPTED` / `REJECTED` / `EXPIRED` / `INVALID`) · o singură cale de execuție (accept cheamă `updateAppointment`) · contul de serviciu `AGENT`, fără permisiune de execuție.

1. **Bază.** `V5__reschedule_proposals.sql`; `NEEDS_RESCHEDULE` în `AppointmentStatus`. Contul de serviciu `AGENT` NEW îl face adminul prin `POST /api/users`.
2. **Time-off urgent.** `POST /api/barbers/{id}/time-off?force=true` (`SCHEDULE_OVERRIDE`): în aceeași tranzacție salvează time-off-ul și trece programările `BOOKED` / `CONFIRMED` suprapuse în `NEEDS_RESCHEDULE`. Fără `force`, regula din F1 rămâne: 409.
3. **Service.** Propunere → aprobare de un om → aceeași cale de execuție NEW. `ProposalService.propose(appointmentId, proposedStart, reason, messageForCustomer)`: verifică slotul cu aceeași validare ca POST-ul (fără să-l ocupe) → `PROPOSED` sau `INVALID` cu `rejection_reason` (`BARBER_NOT_WORKING`, `SLOT_TAKEN`, `IN_PAST`). accept → `AppointmentCommandServiceImpl.updateAppointment(appointmentId, {startTime})`; `NEEDS_RESCHEDULE` redevine `BOOKED`. Slot ocupat între timp → 409, propunerea `EXPIRED`. reject → `REJECTED`, programarea rămâne de reprogramat. **Nu se scrie altă cale de execuție.**
4. **Intrare.** Tool-uri MCP, inclusiv primul de scriere NEW: `getAppointmentsToReschedule()` (`APPOINTMENT_VIEW`), `proposeReschedule(...)` (`PROPOSAL_CREATE`), `getProposal(id)`. REST: `GET /api/proposals?status=` (`PROPOSAL_VIEW`), `POST /api/proposals/{id}/accept`, `POST /api/proposals/{id}/reject` (`PROPOSAL_DECIDE`).
5. **Test.** Cu login real: `AGENT` propune, clientul acceptă; `AGENT` care încearcă accept → 403; alt client → 403.

Reguli

- O singură propunere `PROPOSED` per programare (a doua → 409).
- Agentul nu are `APPOINTMENT_RESCHEDULE` și nici `PROPOSAL_DECIDE`: doar propune.
- Propunerea se face pe același frizer (`updateAppointment` nu schimbă frizerul); procedurile și durata rămân.

Gata când: pe EC2, cu D = o marți din viitor, la cel puțin 2 săptămâni după ziua în care faci F7, ca F8 și U4 să o prindă tot în viitor (de exemplu: F7 pe 2026-12-01 → D = marți 2026-12-15, D + 1 = miercuri 2026-12-16). D și D + 1 sunt zile de program (09:00–17:00 din seeder), nu sunt luni (deci în afara L2, D3, seriei din F4 și D6) și nu sunt vineri (pauza din seeder), iar frizerul 1 nu are alte programări în ele · frizerul 1, în D: A la 10:00 haircut, B la 11:00 Haircut and Beard, C la 14:00 Beard Trim · time-off D 09:00–17:00 fără `force` → 409; cu `force` → 201, 3 programări `NEEDS_RESCHEDULE` · availability D → `[]` · din MCP Inspector cu token AGENT: `proposeReschedule(A, D + 1 10:00)` → `PROPOSED`; `proposeReschedule(B, D 15:00)` → `INVALID`, `BARBER_NOT_WORKING`; a doua propunere pentru A → 409 · `proposeReschedule` cu token de client → eroare de acces · AGENT face accept → 403; B face accept pe propunerea lui A → 403 · A acceptă → programarea A: D + 1 10:00–10:30, `BOOKED`, propunerea `ACCEPTED`.

F8 Asistentul de reprogramare pe reguli — primul microserviciu NEW

AGENT

O aplicație separată, `booking-agent`, verifică la fiecare 5 minute prin MCP ce programări au rămas fără frizer, aplică regulile și propune o oră nouă pentru fiecare. Nu are bază de date și nu cunoaște API-ul REST, doar tool-urile.

Pornești de la: tool-urile din F6–F7, contul `AGENT` din F7, logarea cu `runId` din F4 · NEW client MCP într-o a doua aplicație
Lecția de predat înainte: L7 „Al doilea proiect Spring: client MCP" — proiect nou `booking-agent` lângă `booking-api` · clientul MCP cu tokenul AGENT în header · interfața `ReschedulePlanner` + teste unitare per regulă · Dockerfile propriu și job CI filtrat pe `paths`.

1. **Proiect.** `booking-agent/`, al doilea proiect Spring din repo NEW: Boot 4.1.1, Java 21, Spring AI 2.0.2 cu starter-ul de client MCP NEW. La pornire se loghează ca `AGENT` (credențiale din `.env`) și trimite tokenul la `/mcp`; se reloghează când tokenul mai are sub 5 minute până la `exp` sau la primul 401 de la `/mcp`, apoi reia apelul o singură dată.
2. **Calcul pur.** `ReschedulePlanner` și `RuleBasedPlanner`: o metodă per regulă, teste unitare pe fiecare, inclusiv exact cazul din „Gata când" (B → 11:00, C → 14:00, nu 10:30 / 13:00).
3. **Bucla.** `@Scheduled` la 5 minute: `getAppointmentsToReschedule` → pentru fiecare fără propunere activă: `getAvailability` pe cele 7 zile de după ziua programării (D + 1 … D + 7) → planner → `proposeReschedule`. `runId` în MDC.
4. **Fără migrare:** agentul nu are bază, iar API-ul nu se schimbă.
5. **Ops.** Serviciul `agent` în compose, profilul `agent` (pe EC2 `COMPOSE_PROFILES=core,obs,agent`); job `agent` în Actions, pornit doar de `booking-agent/**` (`paths` NEW).

Reguli

- Regula 1: la același frizer, în zilele D + 1 … D + 7 (D = ziua programării), prima zi care are un slot liber în ± 60 min de ora inițială; în ziua aceea, slotul cel mai apropiat de ora inițială (la egalitate, cel mai devreme). Regula 2: altfel, primul slot liber din D + 1 … D + 7. Altfel nicio propunere și un WARN.
- Mesajul pentru client e un șablon fix (ziua, ora, motivul).
- Agentul oprit nu blochează API-ul.
- Tokenul expirat nu oprește agentul.

Gata când: pe EC2, pornind de la starea din F7 (aceleași D și D + 1, încă în viitor; B și C încă `NEEDS_RESCHEDULE`; în D + 1 doar A, 10:00–10:30): în cel mult 5 minute apar 2 propuneri `PROPOSED`: B → D + 1 11:00, C → D + 1 14:00 · `GET /api/proposals?status=PROPOSED` cu token admin → 2 · în log filtrezi rularea după `runId` · agentul oprit → API-ul răspunde normal, rezervările merg · agentul pornit de peste 1 oră încă propune (în log nu apare 401) · local, cu `JWT_EXPIRATION` pus pe 2 minute, agentul propune și după 10 minute.

F9 Asistentul cu AI: ora potrivită pentru client NEW

AGENT

Același agent primește un al doilea planner, cu Claude: își cere singur sloturile și tiparul de vizite al clientului și alege ora la care clientul vine de obicei, cu un mesaj scris pentru el. Din configurare alegi care planner rulează.

Pornești de la: `ReschedulePlanner` și `RuleBasedPlanner` din F8, clientul MCP din F8, `getCustomerVisitPattern` din F6, `ProposalService` din F7 · NEW Spring AI `ChatClient` cu Claude (`spring-ai-starter-model-anthropic` 2.0.2)
Lecția de predat înainte: L8 „Spring AI ChatClient cu Claude" — `ChatClient` și prompt-ul de sistem din `resources` · tool-urile MCP date modelului · ieșire structurată în record (`appointmentId`, `proposedStart`, `reason`, `messageForCustomer`) · timeout și fallback pe reguli · tokenii și latența ca metrici.

1. **Bază (API).** `V__proposal_planner.sql`: `planner` și `model_latency_ms` pe `reschedule_proposals`; `proposeReschedule` primește `planner`.
2. **Planner.** `LlmPlanner`: `ChatClient` NEW cu tool-urile MCP date modelului NEW, ieșire structurată în record NEW. Prompt-ul de sistem NEW în `resources`: același frizer, aceleași proceduri, doar sloturi din `getAvailability`, mesaj scurt în română pentru client.
3. **Siguranță.** Timeout 20 s și fallback NEW. Fără cheie, fără răspuns sau răspuns care nu se parsează → `RuleBasedPlanner` și un WARN. O oră inventată de model ajunge la API, care o marchează `INVALID`: nu se execută nimic.
4. **Config.** `spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}`, modelul din `${ANTHROPIC_MODEL}`; cheia în `.env` local și ca secret pentru EC2, niciodată în repo. `booking.agent.planner=RULES|LLM`.
5. **Metrici.** Tokeni, latență și propuneri pe planner în Prometheus.
6. **Comparația.** Același scenariu demo (20 de programări `NEEDS_RESCHEDULE` la 4 frizeri, clienți cu istoric), fiecare planner pe rând: % propuneri valide, % în ora obișnuită a clientului ± 1 h, întârzierea medie în zile, latența, costul.

Reguli

- Modelul doar propune: aceeași poartă din F7, același `INVALID` cu motivul.
- Fallback-ul pe reguli e obligatoriu și se vede în log.

Gata când: tabelul comparației reguli vs. AI completat din `reschedule_proposals` (`GROUP BY planner`) și din Grafana · `ANTHROPIC_API_KEY` scos → agentul trece pe reguli, WARN în log, propunerile au `planner = RULES` · o propunere LLM pe o oră din time-off → `INVALID`, programarea neatinsă.

III. Evenimente și servicii

Salonul trece de la „cine cheamă pe cine” la „cine ascultă ce”. Kafka, `booking-notifications` și ELK nu încap pe EC2-ul obișnuit: rulează local sau pe instanța de demo pentru susținere.

F10 Programarea anunță, ceilalți reacționează NEW

CUSTOMER · AGENT

Fiecare schimbare a unei programări și fiecare propunere devin evenimente; clientul primește o notificare, iar agentul pornește imediat ce o programare rămâne fără frizer, nu la 5 minute.

Pornești de la: `AppointmentCommandServiceImpl` (add, update, cancel, confirm, complete), time-off-ul urgent și `ProposalService` din F7, bucla din F8 · NEW Kafka

Lecția de predat înainte: L9 „Kafka în monolit” — broker KRaft în compose și topicurile create de `kafka-init` · `KafkaTemplate` și publicarea după commit (`ApplicationEventPublisher` + `@TransactionalEventListener AFTER_COMMIT`) · `@KafkaListener` și consumer group · consumator idempotent (UQ pe `event_id`) · comutatorul `booking.events.enabled` cu `@ConditionalOnProperty` .

1. **Bază.** `V7__notifications.sql` , cu UQ pe `event_id` (consumator idempotent NEW).
2. **Ops.** `apache/kafka` , un nod KRaft NEW , profilul `messaging` ; `kafka-init` NEW creează `booking.appointment-events` și `booking.proposal-events` (cheie = `appointmentId`). Deploy-ul pe EC2 pornește doar profilurile care încap (`COMPOSE_PROFILES` din `.env` , fără `messaging` și `logs`).
3. **Evenimente.** După commit (`ApplicationEventPublisher` + `@TransactionalEventListener(AFTER_COMMIT)` NEW , apoi `KafkaTemplate` NEW): `AppointmentEvent {eventId, type BOOKED / RESCHEDULED / CANCELED / CONFIRMED / COMPLETED / NEEDS_RESCHEDULE, appointmentId, customerId, customerEmail, barberId, startTime}` ; `ProposalEvent {eventId, type PROPOSED / ACCEPTED / REJECTED, proposalId, appointmentId, customerId, customerEmail}` ; ambele cu `customerEmail` , cheia notificării (F11, F13).
4. **Ascultători, în monolit.** `NotificationListener` (`@KafkaListener` NEW , consumer group propriu) scrie o notificare per eveniment pentru client. Agentul ascultă `NEEDS_RESCHEDULE` și `CANCELED` (un slot eliberat poate ajuta o programare în așteptare) în loc de timer. Publicarea și ascultătorii stau sub `booking.events.enabled` (`@ConditionalOnProperty` NEW ; true doar unde rulează profilul `messaging`). Pe EC2 e false: API-ul nu publică, iar agentul rămâne pe `@Scheduled` la 5 minute; local sau pe demo e true și agentul trece pe `@KafkaListener` .
5. **Intrare.** `GET /api/notifications/me` (`NOTIFICATION_VIEW`).
6. **Test.** IT cu Kafka din Testcontainers: același eveniment de două ori → o notificare.

Reguli

- Un eveniment se publică doar după commit; un rollback nu trimite nimic.
- Același `eventId` de două ori → un singur rând (UQ); duplicatul se ignoră, nu e eroare.

Gata când: local / pe instanța de demo: clientul A rezervă → `GET /api/notifications/me` → 1 notificare `BOOKED` · același eveniment retrimis din consolă → tot 1 · listener oprit, două rezervări, listener pornit → apar ambele · time-off urgent → agentul propune în cel mult 10 s, nu la 5 minute.

F11 Notificările pleacă din monolit NEW

CUSTOMER

Notificările clienților devin serviciul lor, `booking-notifications` , cu baza lor; între el și API singurul contract sunt topicurile.

Pornești de la: `NotificationListener` și `V7` din F10, topicurile din F10, filtrul JWT din F3 · NEW al doilea serviciu desprins, topicurile = contractul

Lecția de predat înainte: L10 „Un serviciu desprins” — schema evenimentului ca contract versionat · o bază per serviciu · același JWT validat în serviciul nou · offset-urile grupului la repornire.

1. **Bază.** O bază per serviciu NEW . În `booking-notifications` : `V1__notifications.sql` ; în `booking-api` : `V8__drop_notifications.sql` .
2. **Proiect.** `booking-notifications/` : Boot 4.1.1, Kafka, JPA, Flyway, baza `booking_notifications_db` ; `NotificationListener` mutat aici, neschimbat; grupul lui își ține offset-ul la repornire NEW .
3. **Intrare.** `GET /api/notifications/me` validează tokenul din F3 (aceeași `JWT_SECRET_KEY` din `.env`); `/me` filtrează după `recipient_email` = claim-ul email (`sub`), iar `NOTIFICATION_VIEW` se citește din claim-ul `permissions` .
4. **Contract.** Evenimentul ca contract versionat NEW : `booking-docs/events.md` : câmpurile `AppointmentEvent` și `ProposalEvent` , cu versiune.
5. **Ops.** Serviciu nou în compose (`messaging`), job CI pe `booking-notifications/**` .

Reguli

- API-ul nu cheamă direct serviciul de notificări; serviciul nu citește baza API-ului.
- Trimiterea reală (SMS, email) nu face parte din proiect: notificarea se salvează și se loghează.

Gata când: API-ul fără `booking-notifications` pornit merge (rezervările trec) · pornit după → recuperează evenimentele din offset · pentru scenariul din F10, aceleași notificări ca în F10 · `GET /api/notifications/me` pe serviciul nou cu tokenul din login → 200; fără token → 401.

F12 Urmăresc o reprogramare prin tot sistemul NEW

ADMIN (în Grafana și Kibana)

De la time-off-ul urgent până la notificarea clientului, o singură reprogramare se vede ca un drum: API → Kafka → agent → MCP → Claude → propunere → notificare → acceptare.

Pornești de la: logurile din F4, metricile din F5, agentul din F8–F9, serviciul din F11 · NEW OpenTelemetry + Tempo + ELK (Elasticsearch, Kibana, Filebeat)

Lecția de predat înainte: L11 „Tracing și loguri centrale” — micrometer-tracing cu OTLP spre Tempo · propagarea prin headerele Kafka și prin apelurile MCP · `traceId` în MDC · Filebeat → Elasticsearch → Kibana.

1. **Trace-uri.** OpenTelemetry NEW în `booking-api` , `booking-agent` și `booking-notifications` , export OTLP în Tempo NEW ; contextul trece prin headerele Kafka și prin apelurile MCP NEW ; Tempo ca sursă de date în Grafana.
2. **Loguri.** Filebeat NEW citește logurile JSON ale containerelor și le trimite în Elasticsearch NEW ; în Kibana NEW cauți după `runId` , `appointmentId` sau `traceId` .
3. **Fără migrare.**
4. **Ops.** Profilul `logs` (Tempo, Elasticsearch, Kibana, Filebeat). Elasticsearch cere 1–2 GB: local sau pe instanța de demo.

Reguli

- `traceId` pe fiecare linie de log a celor trei aplicații.
- Pe EC2-ul mic rulează doar `core` + `obs` + `agent`; ELK nu. Exportul OTLP spre Tempo stă sub același tip de comutator (true doar unde rulează profilul `logs`); pe EC2 e oprit, ca aplicațiile să nu trimită spre un Tempo care nu există.

Gata când: faci un time-off urgent, iei din Kibana `traceId`-ul liniei `NEEDS_RESCHEDULE` și în Tempo vezi un singur drum: API → Kafka → agent → MCP `getAvailability` → Claude → `proposeReschedule` → Kafka → `booking-notifications` · după acceptul clientului, același `appointmentId` apare în Kibana în toate cele trei aplicații.

IV. Platforma, ca la cars

Ultimele două piese ale unei platforme reale: identitatea scoasă din aplicație și o singură poartă de intrare. Tot pe Docker.

F13 Cine ești, spus de Keycloak NEW

toți · AGENT · ADMIN

Conturile și parolele nu mai stau în API: Keycloak emite tokenurile pentru clienți, frizeri și admin, pentru agent fără parolă și pentru UI-urile de administrare.

Pornești de la: matricea de permisiuni din F3, `Role`, filtrul JWT din F3, contul `AGENT` din F7–F8, `booking-notifications` din F11 · NEW Keycloak (OAuth2 / OIDC)

Lecția de predat înainte: L12 „Keycloak" — realm JSON în Git (`--import-realm`) · API ca resource server pe issuer, cu rolurile din token → aceeași hartă rol → permisiuni · client credentials pentru agent · Keycloak și oauth2-proxy publicate prin nginx (`/auth`, `/grafana`).

1. **Bază.** `V9__users_keycloak.sql`. Rezolvarea utilizatorului: întâi după `keycloak_id` = `sub`; dacă nu există, după email și se completează `keycloak_id` (legare); abia altfel se creează `users` + `Customer` (doar pentru rolul `CUSTOMER`, din `given_name`, `family_name` și `phone_number`). Tokenurile cu rolul `AGENT` nu creează utilizator local: `proposed_by` = `preferred_username` sau `azp` din token.
2. **Ops.** Keycloak NEW + Postgres-ul lui în compose, profilul `auth` (pe EC2 `COMPOSE_PROFILES=core,obs,agent,auth`); `realm-booking.json` în `booking-ops/keycloak/`, încărcat cu `--import-realm` NEW. **Publicat prin nginx**, ca browserul să ajungă la el NEW: `https://<domeniu>/auth` → `keycloak:8080`, cu `KC_HTTP_RELATIVE_PATH=/auth`, `KC_HOSTNAME=https://<domeniu>/auth` și `KC_PROXY_HEADERS=xforwarded`; fără `ports`, în rețelele `edge` și `backend` (F14). Grafana trece în spatele oauth2-proxy NEW: `https://<domeniu>/grafana` → oauth2-proxy → Grafana (`GF_SERVER_ROOT_URL=https://<domeniu>/grafana/`, `GF_SERVER_SERVE_FROM_SUB_PATH=true`); portul `127.0.0.1:3000` și tunelul din F5 ies. Parola de admin Keycloak, secretul clientului agentului și cele ale oauth2-proxy doar în `.env`.
3. **Realm.** Roluri `CUSTOMER`, `BARBER`, `ADMIN`, `AGENT`. Clienți: `booking-api`, `booking-agent` (client credentials, rol `AGENT`), `oauth2-proxy`, `booking-ui` (public, PKCE, pentru U5). Atributul `phone` e obligatoriu la înregistrare, iar un mapper îl pune în token ca `phone_number` (`customers.phone` rămâne NN). Un mapper pune `permissions` în token; `booking-notifications` poate verifica și doar rolul `CUSTOMER` / `ADMIN` din `realm_access`, cu email-ul drept cheie.
4. **API + notifications.** Resource server NEW pe issuer-ul realm-ului (`iss` = `https://<domeniu>/auth/realms/booking`; cheile citite intern, de la `http://keycloak:8080/auth/realms/booking/protocol/openid-connect/certs`); `/api/auth/register`, `/api/auth/login` și generatorul de JWT ies. `PUT /api/users/{id}/password` și câmpul `password` din `BarberCreateRequest` / `UserCreateRequest` ies în același commit cu V9; contul de frizer și de admin se face în Keycloak cu același email ca profilul creat prin `POST /api/barbers`, iar legarea se face la primul login.
5. **Agentul.** Token prin client credentials NEW, cerut intern de la `http://keycloak:8080/auth/realms/booking`.
6. **Test.** Tokenuri de la un Keycloak din Testcontainers, pornit cu același realm.

Reguli

- Matricea de permisiuni nu se schimbă; se schimbă doar cine emite tokenul.
- U5 se face imediat după F13, ca UI-ul să nu rămână fără login.

Gata când: pe `https://<domeniu>`, cu token din Keycloak: clientul A → programările lui 200, ale lui B 403 · agentul cu client credentials → `proposeReschedule` merge, accept → 403 · `/api/auth/login` vechi → 404 · după primul login al lui A, `SELECT keycloak_id FROM users WHERE email = A` → completat, același `id` ca înainte · `https://<domeniu>/auth/realms/booking/.well-known/openid-configuration` → 200, cu `issuer` pe același domeniu · `https://<domeniu>/grafana` fără login → redirect pe `https://<domeniu>/auth/realms/booking/…`, pagina de login Keycloak; cu un cont `ADMIN` → Grafana; un utilizator fără rolul `ADMIN` → 403.

F14 O singură poartă: Kong NEW

toți

Aplicațiile nu mai sunt expuse direct: tot traficul, inclusiv al agentului, trece printr-un API gateway care verifică tokenul, limitează abuzul și pune un id pe fiecare cerere.

Pornești de la: `booking-ops/compose.yaml`, tokenurile din F13, MDC-ul din F4, trace-urile din F12 · NEW Kong (API gateway DB-less) + rețele Docker

Lecția de predat înainte: L13 „Kong și rețelele Docker" — `kong.yml` declarativ în Git · plugin-urile `jwt`, `rate-limiting`, `correlation-id`, `prometheus` · rețelele `edge` și `backend`, aplicațiile fără `ports`.

1. **Ops.** Kong DB-less NEW (pe EC2 `COMPOSE_PROFILES=core,obs,agent,auth,edge`), `kong.yml` în `booking-ops/kong/`: `/api/notifications` → `booking-notifications`, `/api` și `/mcp` → `booking-api`.
2. **Plugin-uri** NEW. `jwt` cu cheia publică a realm-ului · `rate-limiting` (de exemplu 60 de cereri pe minut pe availability) · `correlation-id` (ajunge în MDC și în trace) · `prometheus`.
3. **Rețele** Docker NEW. `edge`: nginx, Kong, oauth2-proxy, ui · `backend`: aplicațiile, MySQL, Kafka, Keycloak; în ambele doar Kong, oauth2-proxy și Keycloak; API-ul, agentul și notifications fără `ports`. nginx: `/` → ui, `/api` și `/mcp` → Kong, `/auth` → Keycloak (din F13), `/grafana` → oauth2-proxy (din F13).
4. **Agentul** apelează `/mcp` prin Kong; ui-ul trimite `/api` spre Kong.
5. **Fără migrare.**

Reguli

- Pe EC2 deschise doar 80 și 443, plus 22 doar de la IP-ul lui Rareș (ssh de mână); deploy-ul din F2b nu folosește ssh: EC2 își trage singur imaginile.
- Kong verifică tokenul; permisiunile rămân în API.

Gata când: portul API-ului încercat direct pe EC2 → nu răspunde · portul 22 din altă rețea decât a lui Rareș → timeout, iar un push pe `main` ajunge totuși pe EC2 (F2b) · prin Kong fără token → 401 de la Kong, cu token → 200 · a 61-a cerere pe minut pe availability → 429 · pe EC2: id-ul de corelare din răspuns apare în `docker compose logs api` (linia JSON, câmpul din MDC); local / pe instanța de demo, cu profilul `logs` : același id în Kibana și în Tempo.

V. Interfața — React (booking-client)

Se construiește în paralel cu backend-ul: fiecare U pornește doar după feature-ul de care are nevoie. Pornești de la ce ai făcut în `rares-user-management-local-ui` (`useState`, `map` / `filter`).

U1 Îmi aleg ora din browser NEW

client, fără login încă · după F1

Pe o pagină: alegi frizerul, bifezi procedurile, alegi ziua, vezi sloturile libere și rezervi cu un click; dacă slotul s-a ocupat între timp, vezi mesajul.

Pornești de la: `rares-user-management-local-ui` (`useState`, `map` / `filter`), `GET /api/barbers`, `GET /api/barbers/{id}/procedures`, `GET /api/appointments/availability`, `POST /api/appointments` din F1 · NEW date de la server în React

Lecția de predat înainte: R1 „Date de la server: fetch + useEffect" — `useEffect` și dependențele lui · `fetch` GET și POST cu JSON · stările loading / error · proxy-ul Vite spre API (fără CORS).

1. **Proiect.** Vite + React + TypeScript în `booking-client/`, o singură componentă, ca la user-management-local.
2. **Date.** La schimbarea frizerului, a procedurilor sau a zilei, `useEffect` NEW cere sloturile cu `fetch` NEW (prin proxy-ul Vite NEW); `customerId = 1` din seeder (până la U2).
3. **Rezervare.** Click pe slot → `fetch` POST cu JSON; stările loading / error NEW; 201 → mesaj și sloturile cerute din nou; 409 / 400 → `message` din `ApiError`.

Reguli

- Nicio regulă de domeniu în UI: sloturile și erorile vin doar din API.

Gata când: local, luni viitoare, frizerul 1, Haircut + Beard Trim → 30 de sloturi · click pe 10:00 cu Haircut → mesaj de confirmare, iar pentru Haircut + Beard Trim lista are 26 de sloturi, fără reîncărcarea paginii · a doua rezervare din alt tab pe același slot → mesajul de 409.

U2 Contul meu și programările mele NEW

CUSTOMER · BARBER · după F3

Te loghezi; clientul își vede și își anulează programările, frizerul își vede ziua și confirmă / finalizează.

Pornești de la: U1, `/api/auth/login`, `/api/auth/me`, `/api/appointments/me`, PATCH confirm / cancel / complete din F3, Dockerfile și compose din F2b · NEW componente cu props

Lecția de predat înainte: R2 „Componente cu props" — componenta împărțită în bucăți cu props · funcții trimise prin props (copilul anunță părintele) · afișare condiționată după rol · tokenul ținut în state și pus în header-ul fiecărui `fetch`.

1. **Componente.** `LoginForm`, `BookingForm` (din U1), `MyAppointments`, `BarberDay`, fiecare cu props NEW; copilul anunță părintele printr-o funcție primită în props NEW.
2. **Login.** `POST /api/auth/login` → token în state, pus în header-ul `Authorization` al fiecărui `fetch` NEW; `/api/auth/me` dă rolul; 401 → înapoi la login.
3. **Ops.** Dockerfile în 2 etape (Node → nginx static), serviciul `ui` în compose (profilul `core`); `/api` spre API prin nginx-ul din container. Job `ui` în Actions pe `booking-client/**`.

Reguli

- 403 și 404 apar ca mesaje din `ApiError`, nu ca pagină goală.

Gata când: pe EC2: clientul A se loghează, rezervă luni 10:00 și o vede în „Programările mele" · frizerul 1 se loghează, apasă Confirmă → la A statusul devine `CONFIRMED` după reîmprospătare · parolă greșită → mesajul de 401 · clientul nu vede butonul Confirmă.

U3 Ziua salonului, pe pagini NEW

ADMIN · CUSTOMER · BARBER · după F5

Aplicația are pagini cu adrese: rezervare, programările mele, ziua frizerului și, pentru admin, ziua salonului cu ocuparea fiecărui frizer, actualizată singură.

Pornești de la: U2, `GET /api/appointments?barberId=&date=` din F1, `GET /api/admin/occupancy?date=` din F5 (`OCCUPANCY_VIEW`) · NEW React Router

Lecția de predat înainte: R3 „Pagini cu React Router" — rute și `Link` · `useParams` și `useNavigate` · ruta protejată după rol · reîmprospătare cu `setInterval` în `useEffect`, oprită la ieșirea din pagină.

1. **Rute** cu React Router NEW (`Link`, `useParams`, `useNavigate`): `/book`, `/me`, `/barber/day`, `/admin/day`; ruta protejată NEW trimite la `/login` fără token și ascunde ce nu permite rolul.
2. **Ziua salonului.** Ocuparea fiecărui frizer și programările zilei, reîmprospătate la 30 s (`setInterval` în `useEffect`, oprit la ieșirea din pagină NEW).

Reguli

- Meniul arată doar ce permite tokenul; o rută tastată de mână fără drept → mesajul de acces interzis.

Gata când: pe EC2, în ziua în care faci U3 (zi lucrătoare, înainte de 09:00, fără alte programări la frizerul 1), refaci scenariul din F5: haircut 09:00 + Haircut and Beard 11:00 + 14:00 = 120 / 480 → `/admin/day` arată 25% la frizerul 1, ca în Grafana · clientul anulează 11:00 → în cel mult 30 s pagina arată 15,6%, fără reîncărcare · clientul care deschide `/admin/day` → mesajul de acces interzis.

U4 Accept ora propusă NEW

CUSTOMER · ADMIN · după F7 (F8–F9 pentru date reale)

Clientul vede propunerea de reprogramare cu mesajul pentru el și apasă Accept sau Refuz; adminul poate decide în numele clientului.

Pornești de la: U3, `GET /api/proposals`, `POST /api/proposals/{id}/accept|reject` din F7 · NEW test end-to-end Playwright

Lecția de predat înainte: R4 „Test e2e cu Playwright” — `page.goto` și `getByRole` · login în test · `expect` pe text · rularea în Actions după build-ul ui.

1. **Pagini.** În `/me`, propunerile `PROPOSED` ale clientului: ora veche, ora nouă, `messageForCustomer`, Accept / Refuz; 409 la accept → mesaj „ora s-a ocupat”.
2. **Test.** Playwright NEW: login client → Accept → programarea apare la ora nouă.

Reguli

- UI-ul nu decide nimic: acceptarea e doar POST-ul din F7.

Gata când: pe EC2, după F8 și înainte de D + 1 (D și D + 1 din F7): clientul B vede propunerea lui din F8, încă `PROPOSED`, pe D + 1 11:00, cu mesajul, apasă Accept → programarea lui e D + 1 11:00 `BOOKED` · testul Playwright verde în Actions.

U5 Login prin Keycloak NEW

toți · imediat după F13, înainte de F14

Formularul propriu de login dispare: te loghezi pe pagina Keycloak și te întorci în aplicație.

Pornești de la: U1–U4, realm-ul și clientul `booking-ui` din F13 · NEW OIDC: authorization code + PKCE

Lecția de predat înainte: R5 „Login OIDC cu PKCE” — fluxul authorization code + PKCE · o bibliotecă OIDC pentru React (`oidc-client-ts` / `react-oidc-context`) · reînnoirea tokenului · logout și în Keycloak.

1. **Realm.** Client public `booking-ui`, cu redirect-urile UI-ului, fără secret: authorization code + PKCE NEW.
2. **UI.** `LoginForm` iese; tokenul vine din biblioteca OIDC NEW (cu reînnoire și logout NEW); header-ul din U2 rămâne.

Reguli

- Niciun secret în codul UI-ului.

Gata când: „Login” duce pe pagina Keycloak și înapoi · clientul și frizerul văd meniuri diferite · logout închide sesiunea și în Keycloak · vechiul formular nu mai există.

Toate cele 15 feature-uri (F1–F14, cu F2b) și cele 5 de interfață fac parte din proiect. F-urile merg în ordine; fiecare U pornește doar după feature-ul scris la el, iar U5 se face între F13 și F14. Niciun feature nu începe înainte de lecția lui.
Versiunile: Spring Boot 4.1.1, Java 21 (din F2), MySQL 8.4, Spring AI 2.0.2 (`spring-ai-bom`). Perechea Spring AI 2.x ↔ Boot 4.x se confirmă cu un build real la începutul F6.
Doar Docker, cu profiluri compose (`core`, `obs`, `agent`, `messaging`, `logs`, `auth`, `edge`). Pe EC2-ul obișnuit: `core` + `obs` + `agent`, apoi `auth` + `edge`, alese prin `COMPOSE_PROFILES` în `.env`. Kafka, `booking-notifications` și ELK cer o instanță mai mare: local sau pe instanța de demo pentru susținere.
Secretele (parola MySQL, `JWT_SECRET_KEY` până la F13, `ADMIN_EMAIL` / `ADMIN_PASSWORD`, credențialele contului AGENT, `ANTHROPIC_API_KEY`, parola de admin Grafana, parola de admin Keycloak, secretul clientului agentului și cele ale oauth2-proxy) stau doar în `.env`, pe EC2 și local. Niciodată în repo, niciodată în compose.

Lecțiile de predat, în ordine

Lecția	Înainte de	Noțiuni
NEW L1a „Baza versionată”	F2	Flyway V1 + `ddl-auto: validate` · config din variabile de mediu · Testcontainers MySQL cu `@ServiceConnection` + failsafe (`mvn verify`) · `@Lock(PESSIMISTIC_WRITE)` + `READ COMMITTED` · testul cu două cereri în paralel (`CompletableFuture` , `@RepeatedTest`)
NEW L1b „Aplicația în container, pe EC2”	F2b	Dockerfile în 2 etape · compose cu healthcheck, profiluri + `.env` · GitHub Actions + GHCR · EC2: security group, ssh · deploy prin pull: `cron` + `deploy.sh`
NEW R1 „Date de la server: fetch + useEffect”	U1 (după F1)	`useEffect` și dependențele · `fetch` GET / POST cu JSON · loading / error · proxy-ul Vite
NEW L2 „Spring Security cu JWT”	F3	`SecurityFilterChain` stateless + BCrypt `PasswordEncoder` · emitere + validare JWT în filtru · permisiunile în service: `Actor` , `access.require` , proprietarul, 401 vs 403 · teste cu login real: helperul login → token + `@TestConfiguration` · nginx + certbot pentru TLS
NEW R2 „Componente cu props”	U2 (după F3)	props · funcții prin props · afișare condiționată după rol · tokenul în state și în header
NEW L3 „Joburi programate și loguri care se pot căuta”	F4	`@Scheduled` + `@EnableScheduling` · `Clock` injectat · Logger SLF4J și niveluri · MDC `runId` / `seriesId` · loguri JSON pe EC2
NEW L4 „Metrici cu Micrometer și Grafana”	F5	Actuator + `/actuator/prometheus` pe portul de management nepublicat · Counter, Gauge, Timer cu tag-uri · Prometheus în compose · Grafana cu dashboard JSON încărcat automat · tunel ssh spre Grafana (până la F13)
NEW R3 „Pagini cu React Router”	U3 (după F5)	rute și `Link` · `useParams` / `useNavigate` · ruta protejată după rol · `setInterval` în `useEffect` cu oprire
NEW L5 „Server MCP cu Spring AI”	F6	MCP: tool-uri, Streamable HTTP · `@Tool` / `@ToolParam` cu descrieri pentru model · MCP Inspector · `/mcp` în spatele filtrului JWT
NEW L6 „Agentul propune, omul decide”	F7	tool MCP de scriere care doar salvează · statusurile propunerii · o singură cale de execuție (accept → `updateAppointment`) · contul AGENT fără drept de execuție
NEW R4 „Test e2e cu Playwright”	U4 (după F7)	`page.goto` + `getByRole` · login în test · `expect` pe text · rulare în Actions
NEW L7 „Al doilea proiect Spring: client MCP”	F8	proiectul `booking-agent` · client MCP cu tokenul AGENT · `ReschedulePlanner` + teste unitare per regulă · Dockerfile + job CI pe `paths`
NEW L8 „Spring AI ChatClient cu Claude”	F9	`ChatClient` + prompt din `resources` · tool-urile MCP date modelului · ieșire structurată în record · timeout + fallback pe reguli · tokeni și latență ca metrici
NEW L9 „Kafka în monolit”	F10	KRaft în compose + `kafka-init` · `KafkaTemplate` + `ApplicationEventPublisher` + `@TransactionalEventListener(AFTER_COMMIT)` · `@KafkaListener` și consumer group · idempotență cu UQ pe `event_id` · comutatorul cu `@ConditionalOnProperty`
NEW L10 „Un serviciu desprins”	F11	evenimentul ca contract versionat · o bază per serviciu · același JWT validat în serviciul nou · offset-uri la repornire
NEW L11 „Tracing și loguri centrale”	F12	micrometer-tracing + OTLP spre Tempo · propagare prin Kafka și MCP · `traceId` în MDC · Filebeat → Elasticsearch → Kibana
NEW L12 „Keycloak”	F13	realm JSON cu `--import-realm` · resource server pe issuer + rol din token → aceeași hartă de permisiuni · client credentials pentru agent · Keycloak și oauth2-proxy publicate prin nginx (`/auth` , `/grafana`)
NEW R5 „Login OIDC cu PKCE”	U5 (după F13)	authorization code + PKCE · bibliotecă OIDC pentru React · reînnoirea tokenului · logout în Keycloak
NEW L13 „Kong și rețelele Docker”	F14	`kong.yml` DB-less · plugin-urile `jwt` , `rate-limiting` , `correlation-id` , `prometheus` · rețelele `edge` / `backend` fără `ports`